

Week 12: Coursework and revision

Convergence, testing, documentation, use of ChatGPT...

Dr K Clough, Topics in Scientific computing, Autumn term 2025

Announcements

1. Your coursework marks should now show, along with a marked up pdf (should be added later today) - if you spot addition errors please let me know.
2. Learning support hour this week will again move to **Wednesday between 12-1pm** in the basement of the Maths building.
3. After Christmas I will run a session on **15 January from 2pm-5pm in my office**, where you can come with last minute coursework questions.

This is very close to the deadline (16th January at 5pm) so in the meantime if you have questions **please email me at k.clough@qmul.ac.uk**.

Plan for today

1. Coursework 1 feedback
2. Coursework 2 run through / revision of key concepts
3. Any other questions you have for me

**“Such is the madness of ambition
that you will feel you have come in
last if anyone is ahead of you.”**

Seneca, Epistles

Mark scheme

1. Question 1: 35 marks total =

Masses (1) Initial conditions (2) Classes - Star and StarSystem (5)
Plots (4) Phase diagram (3) Solve_ivp (5) Midpoint (7)
Convergence test (6) Euler versus Midpoint (2)

2. Question 2: 25 marks total =

Stable trajectories working and plots (8) Phase diagram (2)
Convergence (6)
Chaotic plots that show chaos + ejection (5)
Phase diagram and explanation of why it differs (4)

3. Question 3: 10 marks total =

Modification works (3) Implemented as inheritance (1)
Two body plot (2) Three body plot (2) Mercury explanation (2)

Common sources of losing marks

1. What is documentation? How should it be formatted?
2. Asserts but no tests of them
3. Plots not clear
4. Convergence tests
5. Naming and abbreviating variable names - when is it ok?
6. Stability versus accuracy (versus energy conservation)
7. Phase plots?
8. Why look at Mercury?

What is “documentation”?

- Markdown formatting:
<https://www.markdownguide.org/cheat-sheet/>
- The documentation required depends heavily on the expected user. Usually people make the mistake of **assuming too much prior knowledge of the user**.
- Don't just repeat the question back to me in new words!
- Say the obvious things! “So we can see that X is bigger than Y, which is what we expected.”

Why is this not a good start?

Problem 1: Binary system (35 marks)

Model a two body system made up of two stars undergoing multiple stable orbits.

Required Components

1. The masses of the stars should be $M_1 = 0.6$, $M_2 = 0.3$
2. The first star should start at position $[x_1, y_1] = [2.0, 0.0]$ with velocity $\left[\frac{dx_1}{dt}, \frac{dy_1}{dt}\right] = [0.0, -0.1]$. Put the second star at $[x_1, y_1] = [-2.0, 0.0]$ and use equations (3) and (4) above to find its initial velocity so that the stars orbits are fixed with respect to the coordinate axes
3. You should write (at least) one class to represent stars and one for star systems
4. Provide plots of the trajectories for the stars

NB Do not provide animations, as these sometimes don't work on different systems and require additional python packages, you must provide static plots that illustrate the results. It is a useful skill to be able to represent time varying data using static plots.

5. Provide a phase diagram for the x-position and x-velocity of the first star. What does this tell you about the system?
6. Compare the solutions using scipy's `solve_ivp()` and the midpoint method (which you should implement yourself)
7. For the midpoint method, provide a convergence test for the x-position of the second star, confirming that the solution converges at the required order.
8. Give two reasons why it is better to use the midpoint method than Euler's method

(HINT 1: you may want to investigate the `rtol` parameter for `solve_ivp()` when you show multiple orbits)

(HINT 2: try to think ahead - what features of problem 1 might you want to reuse in problems 2 and 3? You can save time coding by making it sufficiently general from the start. Don't forget the naming conventions of the course!)

```
In [ ]: # Your code here - insert more cells including markdown cells as required
```

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# The following lines makes the plots appear inline in the notebook
%matplotlib inline

# Global variables, consider these as hard coded but give them names for readability
DIMENSION = 2 # x and y
G Newton = 1
```


Why is this a good start?

4. Explain why Albert should look at the motion of Mercury, a planet that is small and close to the sun, to see if he can see this modification in observational data, rather than Jupiter, which is massive but further out?

Start of Coursework

Trisolaris: Simulating the 3 Body Problem

This is a great first impression

Author: [Redacted]

Welcome! So, you're curious about the how a system of stars evolves over time. Well, you've come to the right *Jupyter* Notebook. Despite systems with more than 2 stars being very rare, in here, this (artificial) universe is at your disposal! Now, let us get star-ted!

Let us first start, like all good notebooks do, by importing the necessary libraries.

- *NumPy* is a necessity for any scientific computing project. It has many useful functions and their **arrays** make life a lot simpler.
- *matplotlib.pyplot* is a very nice illustrator for *Python*. It allows us to visualise our data and give us a better understanding of what's happening.
- Finally, *SciPy* is a vast library and it is a waste to import all of it, especially if we only wish to use `solve_ivp`, which is one of the most robust integrators around. This will help us solve our system of stars!

```
[ ]: import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
```

Setting up the Stars

Before we can throw stars into our pot, we need to construct them! This is much easier and less time consuming than real life. Here, we will give the stars all the variables they needs:

- Its positions $(\mathbf{r}_t)_{t=0}^T$ at time $t \in [0, T]$, where $\mathbf{r}_t = (x_t, y_t) \in \mathbb{R}^2$
- Its mass $m \in \mathbb{R}_{>0}$
- Its velocities $(\mathbf{v}_t)_{t=0}^T$ at time $t \in [0, T]$, where $\mathbf{v}_t = (v_{xt}, v_{yt}) \in \mathbb{R}^2$
- Its name - a string. You can't just bring this hot ball of fusion into this world and not give it a name.

As you can see, these restrictions on possible values use **asserts** to ensure there is no funny business. We have also ordered the variables like this, so that both the velocities and the name can be given default values: `[np.nan, np.nan]` and `""` respectively. These default values were given so that it would be easy to find and change later on, and sometimes you need more time to ponder (we will see this later, so remember!).

What is “documentation”?

- The **target user** is important - in this case the documentation is for a new student on this course, who may be rusty on maths or python or both.
- Possible components of documentation:

A list of capabilities of the code, and overview of the structure, prerequisites (did you tell the user to active the python environment? What python modules do you need?), list of coding conventions (what order are your variables in the state vector?), what do the parameters do? Do they have expected ranges? What would break the code? How do I know it has worked? What is a phase plot? How does the convergence test work? ...

Other tips on “documentation”

- Makes sense for it to come BEFORE the code it describes
- Use the notebook format wisely, no big blocks of code or main functions, intersperse code and docs
- Judgement required for split of markdown versus inline comments
- Use formatting - line breaks, headings, fonts / code snippets etc
- Answering questions is not documentation
- Headings are not documentation
- Use ChatGPT to polish the English if poor (but NOT to write it)

Asserts and testing

1. Asserts require **judgement** and **humility**. The usual mistake is to underestimate your/the user's ability to screw things up
2. **Each assert should be tested with a unit test.** e.g. if you should not be able to set the mass attribute of the star class negative, create a class inputting a negative mass parameter and check that it triggers the assert. This may seem overkill, but I refer you back to point (2).

```
def update_velocity(self, new_velocity):
    assert np.shape(new_velocity)[-1] == 2, "This is for a 2D star system, so velocity must be a stack of 2-vectors."
    self.v = np.vstack((self.v, new_velocity))

def update_mass(self, new_mass):
    assert np.shape(new_mass) == np.shape(0), "Mass is not a scalar, did you mean to swap the velocity and the mass"
    assert new_mass >= 0, "Probably best to give the star a positive mass."
    self.m = new_mass

def update_name(self, new_name):
    self.name = new_name
```

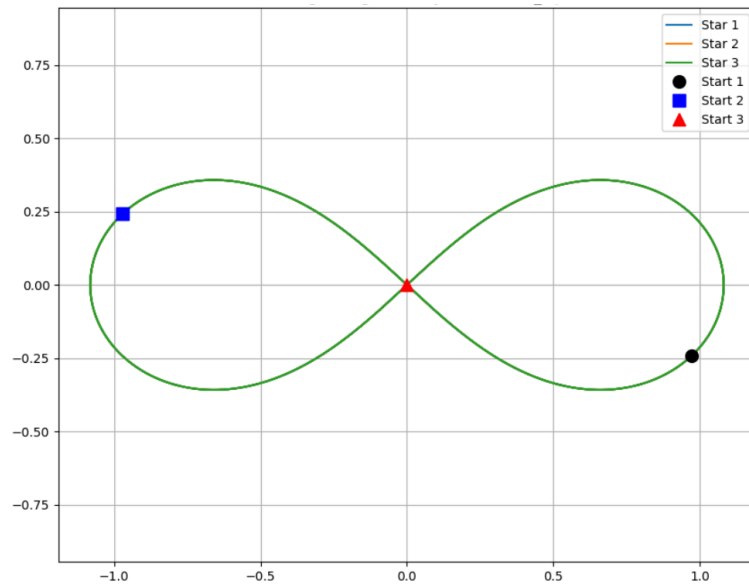
```
'''
    Test Zone!
    Uncomment to see each assert
'''

#Star([1,1,1], 2, [12,11]) #This will assert "This is for a 2D star system, so position must be a 2-vector."
#Star([1,1], 2, [12,11,3]) #This will assert "This is for a 2D star system, so velocity must be a 2-vector."
temp_star = Star([1,2], 2) #This will give the warning: "Warning, velocity has been left blank"
#Star([1,1], [2]) #This will assert "Mass is not a scalar, did you mean to swap the velocity and the mass"
#Star([1,1], -2) #This will assert "Probably best to give the star a positive mass."

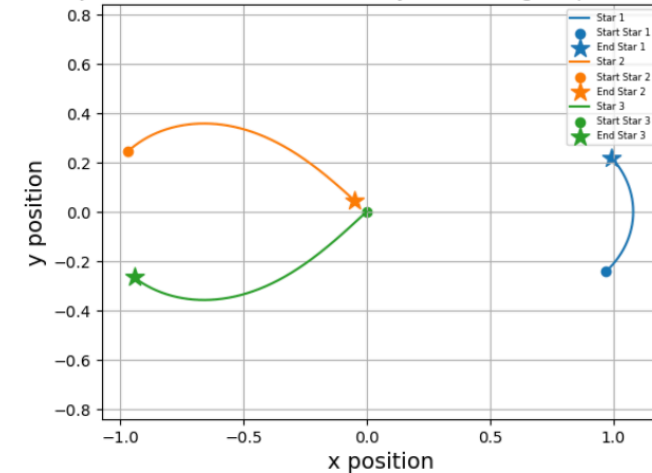
#temp_star.update_mass(-2) #This will assert "Probably best to give the star a positive mass."
#temp_star.update_mass([2]) #This will assert "Mass is not a scalar, did you mean to swap the velocity and the mass"
#temp_star.update_velocity([2]) #This will assert "This is for a 2D star system, so velocity must be a stack of 2-vectors."
#temp_star.update_position([2]) #This will assert "This is for a 2D star system, so position must be a stack of 2-vectors."
```

Plotting

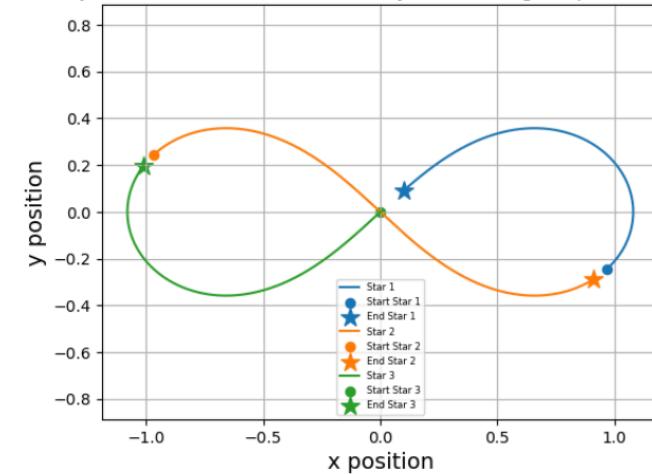
Never make a plot with indistinguishable lines on it.



Trajectories of Stable Three Star System - using Midpoint Method



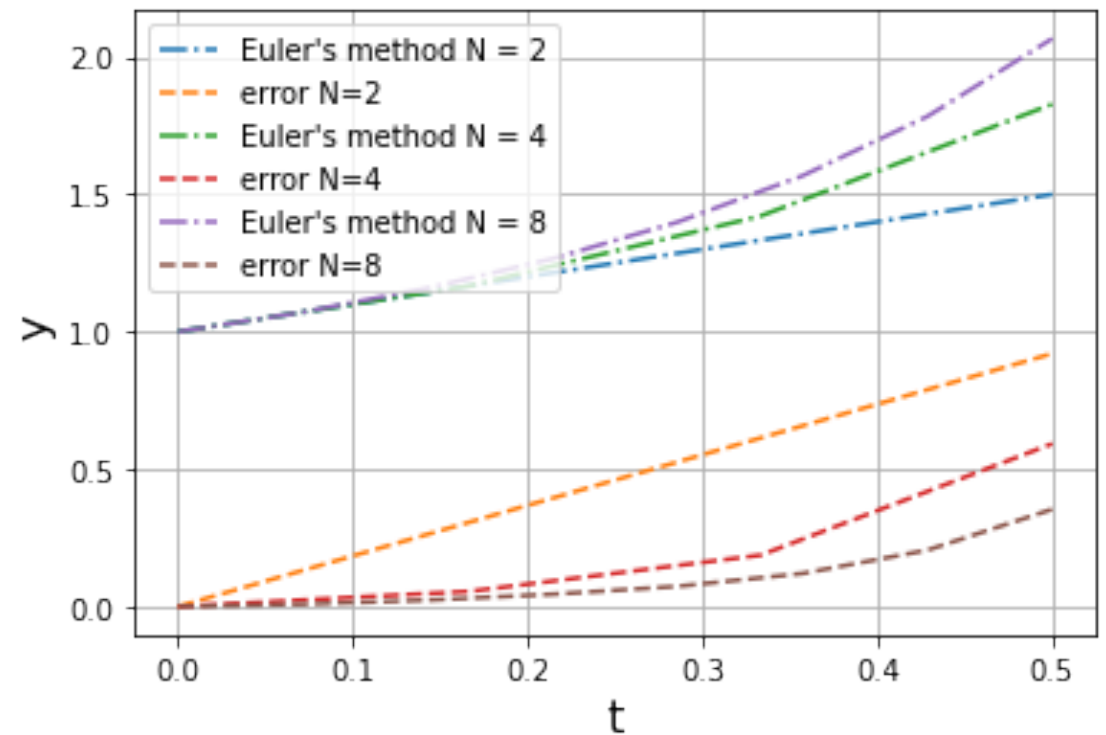
Trajectories of Stable Three Star System - using Midpoint Method



Convergence

Where **we don't know the solution**, we need **3 RESOLUTIONS** to test convergence - if we double the resolution, we know that the differences should scale as

$$\frac{y_{N=8} - y_{N=4}}{y_{N=4} - y_{N=2}} = 1/2$$

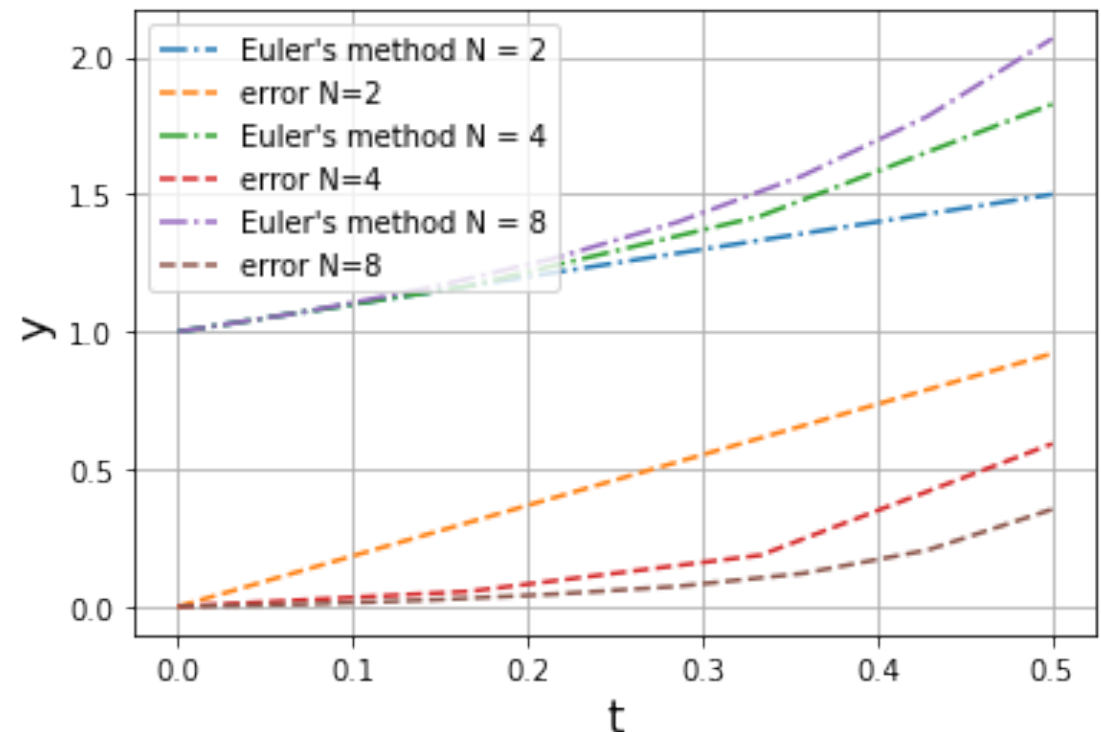


Convergence

You cannot assume a very high resolution (or `solve_ivp`) gives the correct solution, this literally defeats the point of doing a convergence test, it is also unlikely that you can afford it in a realistic scenario

Where **we don't know the solution**, we need **3 RESOLUTIONS** to test convergence - if we double the resolution, we know that the differences should scale as

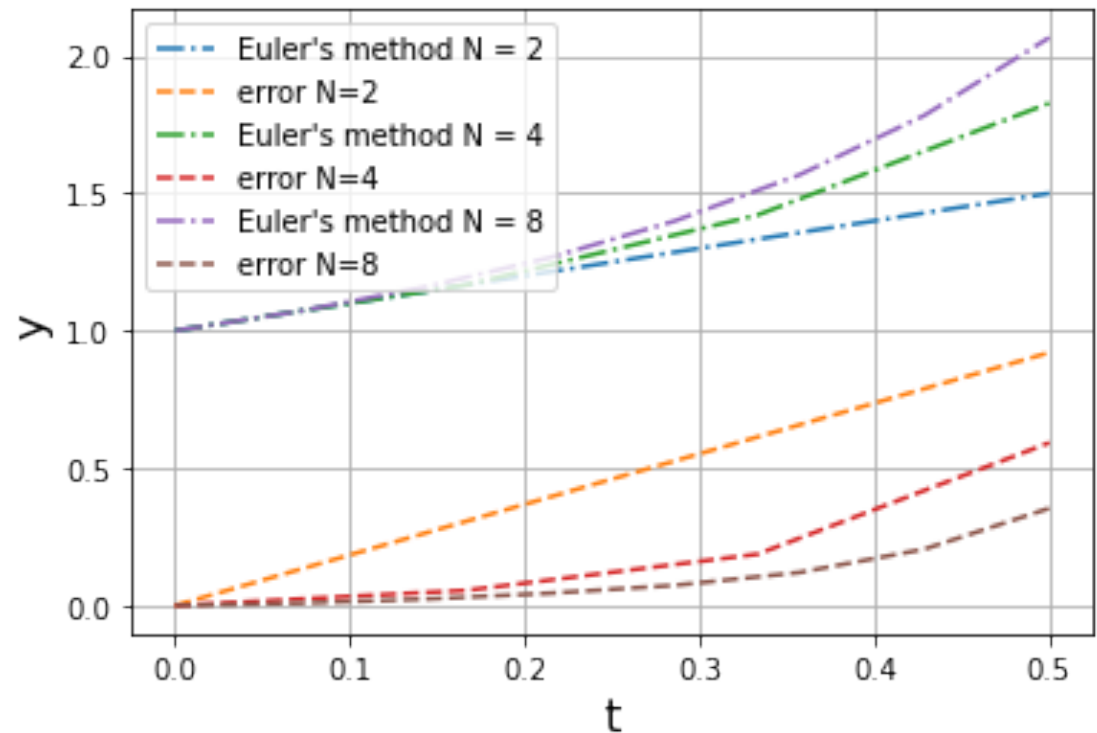
$$\frac{y_{N=8} - y_{N=4}}{y_{N=4} - y_{N=2}} = 1/2$$



Convergence

Because

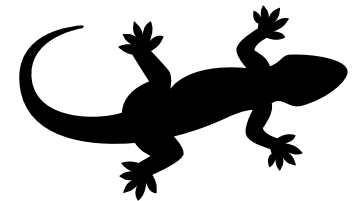
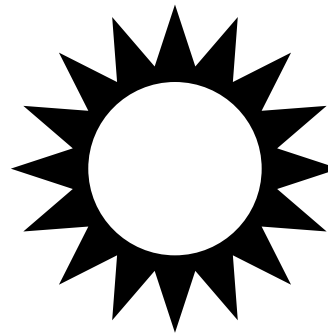
$$\begin{aligned} & \frac{(y_{N=8} - y_{true}) - (y_{N=4} - y_{true})}{(y_{N=4} - y_{true}) - (y_{N=2} - y_{true})} \\ &= \frac{(y_{N=4} - y_{true})/2 - (y_{N=4} - y_{true})}{(y_{N=4} - y_{true}) - 2(y_{N=4} - y_{true})} \\ &= \frac{1}{2} \end{aligned}$$



If you change the resolutions you compare you will change the convergence order

Naming quiz: Which of the following would (in the context of the coursework) be an acceptable name?

1. A bool called mass_is_positive
2. A variable called sol_bin_newt
3. A bool called pos_mass
4. A function called ode()
5. A float called pos_ivp
6. A float called x
7. A class called starsystem



Commenting

```
#Function that calculates the mass given the density and volume
def calculate_mass(density, volume) :
    mass = density * volume
    return mass

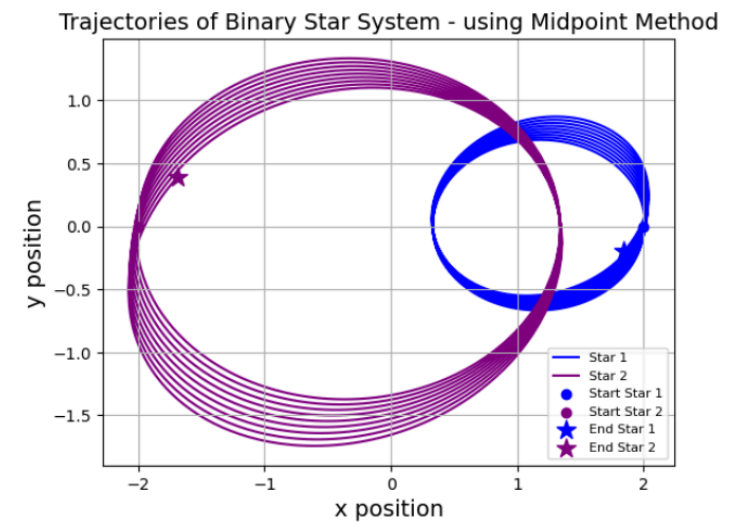
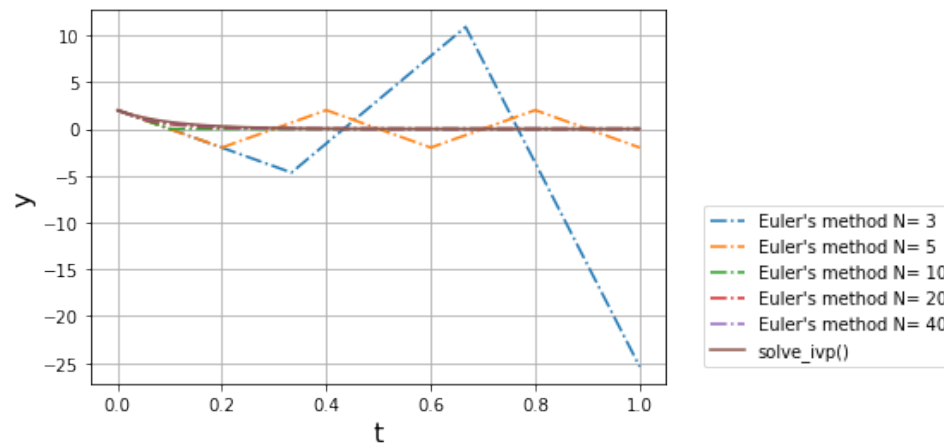
# Assigns the value of 3 to a
a = 3

# Uses Equation (3.2) in Clough et. al. 2022 Phys.Rev.Lett. 129 (2022)
g_tt = E + 0.5 * V_of_phi
```

Q: Which of these comments are useful?

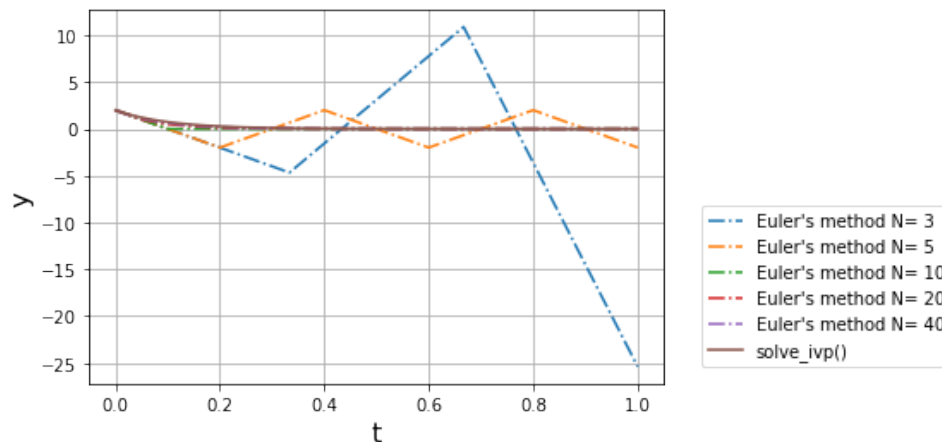
Stability versus accuracy?

Is this stable? Is it accurate?

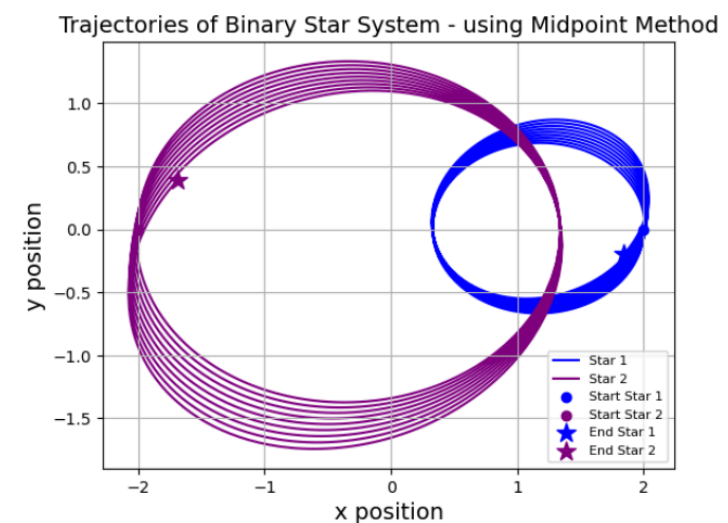


Stability versus accuracy?

Is this stable? Is it accurate?



Unstable

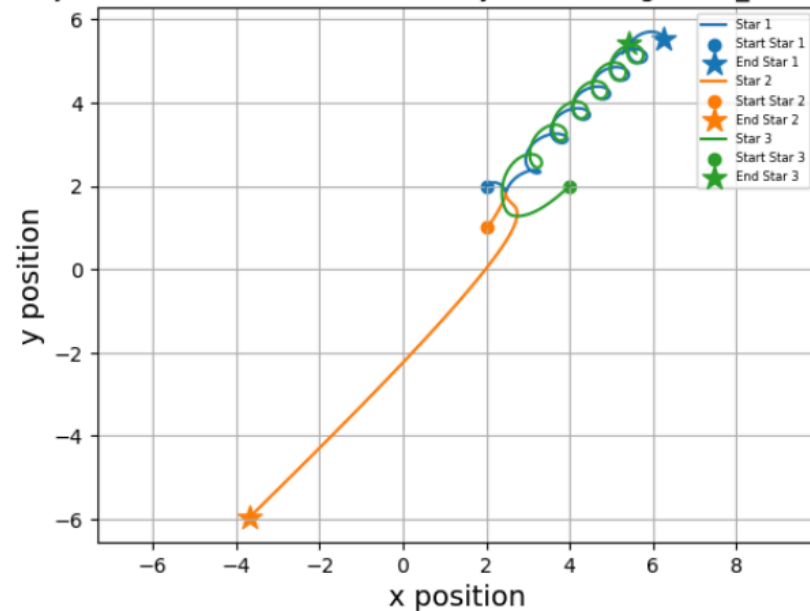


Stable but not accurate
(definitely not preserving energy)

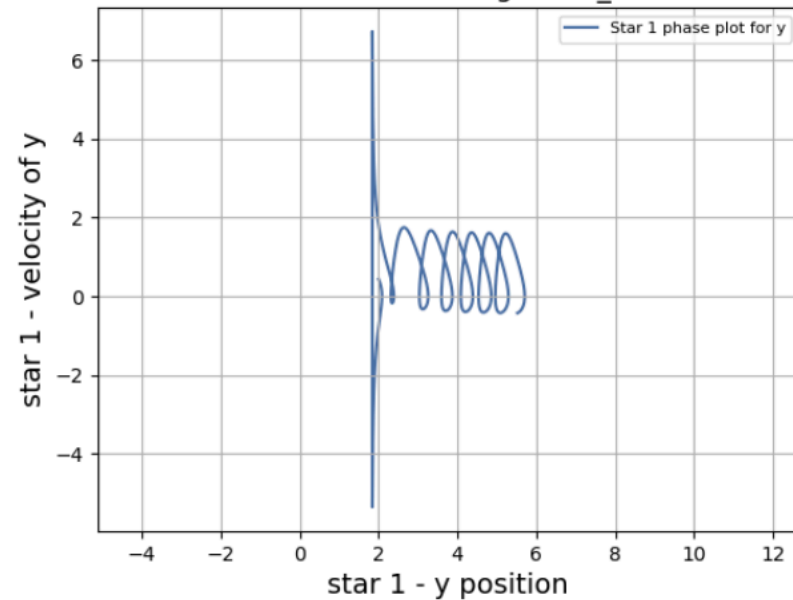
Chaotic phase plots

Which part of the phase plot is chaotic?

Trajectories of Chaotic Three Star System - using Solve_IVP Method



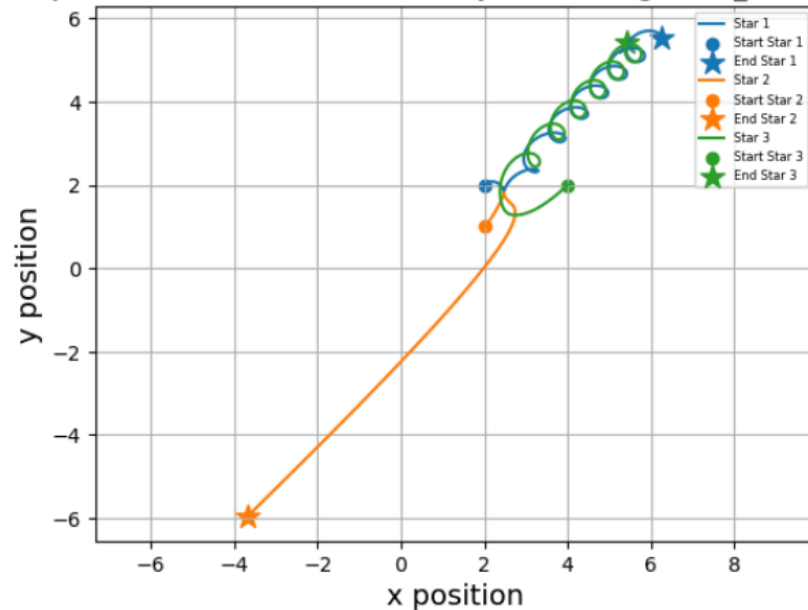
Phase Plot for Star 1 - using Solve_IVP Method



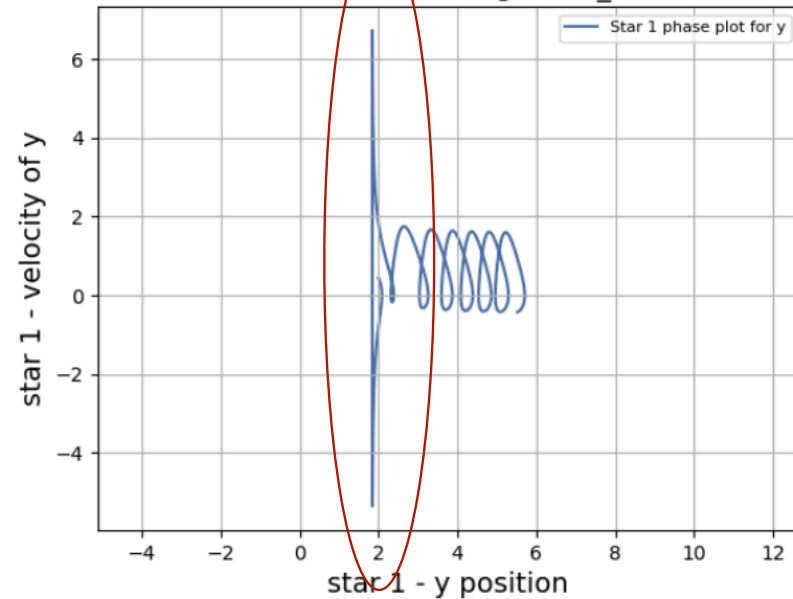
Chaotic phase plots

Which part of the phase plot is chaotic?

Trajectories of Chaotic Three Star System - using Solve_IVP Method



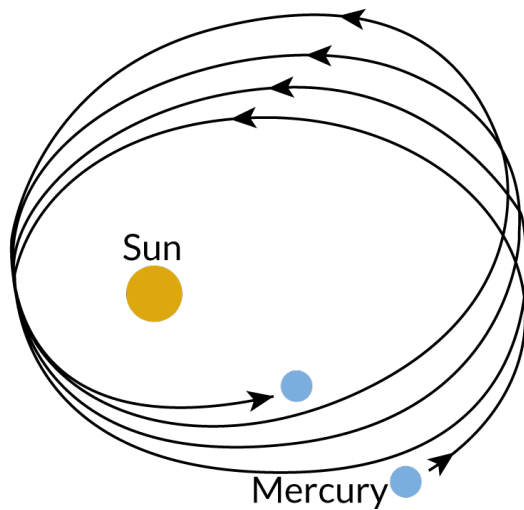
Phase Plot for Star 1 - using Solve_IVP Method



The latter part (once it is in a two body system) is stable

Correction to Mercury?

What matters for *this specific correction*?



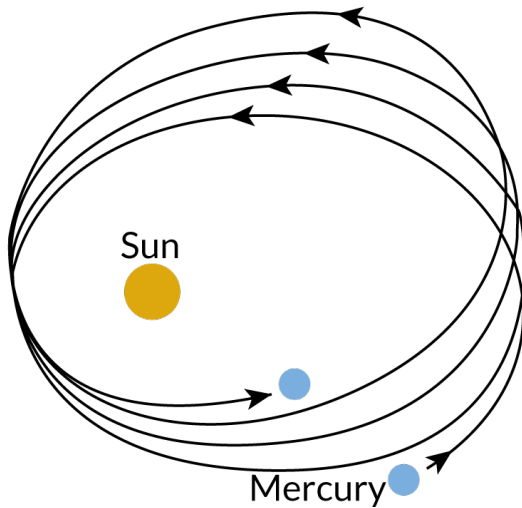
$$\frac{d^2 x_i}{dt^2} = \sum_{j=1, j \neq i}^{j=N} \left[1 + \frac{3\lambda^2 L_{ij}^2}{r_{ij}^2} \right] \frac{M_j}{r_{ij}^2} \cos(\theta) \quad (4)$$

$$\frac{d^2 y_i}{dt^2} = \sum_{j=1, j \neq i}^{j=N} \left[1 + \frac{3\lambda^2 L_{ij}^2}{r_{ij}^2} \right] \frac{M_j}{r_{ij}^2} \sin(\theta) \quad (5)$$

$$\text{where } L_{ij} = \left(\frac{dy_j}{dt} - \frac{dy_i}{dt} \right) (x_j - x_i) - \left(\frac{dx_j}{dt} - \frac{dx_i}{dt} \right) (y_j - y_i)$$

Correction to Mercury?

What matters for *this specific correction*?



$$\frac{d^2 x_i}{dt^2} = \sum_{j=1, j \neq i}^{j=N} \left[1 + \frac{3\lambda^2 L_{ij}^2}{r_{ij}^2} \right] \frac{M_j}{r_{ij}^2} \cos(\theta) \quad (4)$$

$$\frac{d^2 y_i}{dt^2} = \sum_{j=1, j \neq i}^{j=N} \left[1 + \frac{3\lambda^2 L_{ij}^2}{r_{ij}^2} \right] \frac{M_j}{r_{ij}^2} \sin(\theta) \quad (5)$$

$$\text{where } L_{ij} = \left(\frac{dy_j}{dt} - \frac{dy_i}{dt} \right) (x_j - x_i) - \left(\frac{dx_j}{dt} - \frac{dx_i}{dt} \right) (y_j - y_i)$$

Distance, velocity, but not mass (of the orbiting body)

Plan for today

1. ~~Coursework 1 feedback~~
2. Coursework 2 run through / revision of key concepts
3. Any other questions you have for me

Lets look at the coursework

Lets look at the prior year example.

Why is it NOT a good example in terms of documentation, testing or naming?

Plan for today

1. ~~Coursework 1 feedback~~
2. ~~Coursework 2 run through / revision of key concepts~~
3. Any other questions you have for me

Any other questions?